Day 145: Ethical Considerations in Generative AI: Bias, Deepfakes, Real Case Studies

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 21, Day 6 | Complete study guide

Watch: Ethical Concerns with Generative AI (Part 1 of 2) (Douglas Schmidt) | How to spot deepfakes and AI-generated images (New Scientist)

Script: `day145_generative_ai_ethics.py` · every number in this guide is copied by program from `day145_run_output.txt`, the real output of that script.

The models of Days 141–144 draw handwritten digits, which harm nobody. The same methods, scaled up, draw faces, voices and photographs, and three problems follow them: a model can copy or exaggerate the skews in its training data, its output can be passed off as real, and it can reproduce its training images. Each is measured here on digits, where every effect can be counted, and set beside a documented real case. Trained on digits whose ones were skewed to lean right, the model drew right-leaning ones 0.414 of the time against 0.110 for a control trained without the skew, and left-leaning ones 0.024 against 0.217: the rarer kind shrank further than in the data, and guidance shrank it more. A detector of generated digits caught 0.924 of held-out fakes but only 0.041 of a VAE's and 0.061 of a GAN's, and simple edits defeated it. A model trained on 10 digits reproduced them (near copies 0.806); on 1,000, near copies were as rare as for real new digits.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

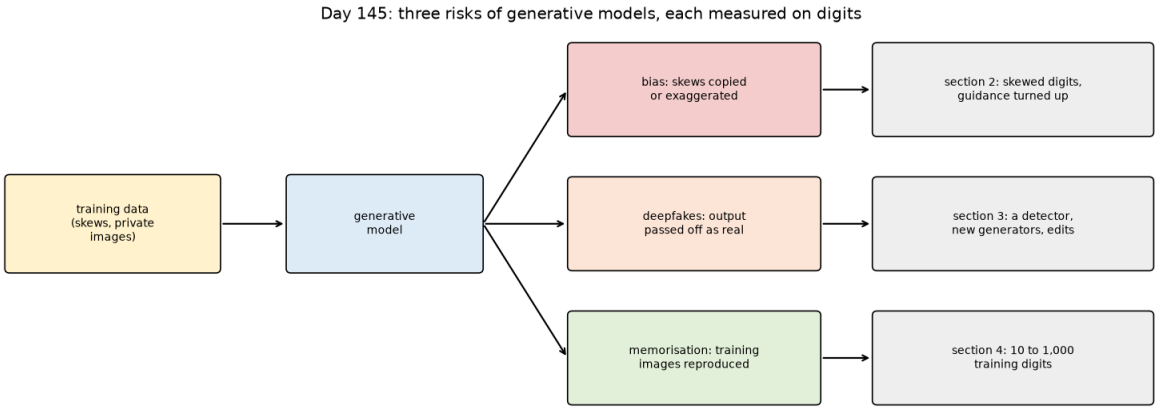
Bias here means a model drawing some kinds of image more or less often than they occur in the world it is meant to represent. A model learns from its training data, so a skew in the data can become a skew in the output, sometimes a larger one.

A **deepfake** is generated media presented as real: a face, a voice, a photograph. A **detector** is a classifier trained to tell generated from real. **Memorisation** is a generative model reproducing training images, which matters when those images are private or copyrighted.

REAL-WORLD SCENARIO

A company uses an image generator for advertising photos of 'a doctor'. Nobody asked for a particular kind of person, but if most photos of doctors the model learned from show one kind, most of its doctors will too, and guidance may make that stronger. The same company's finance team receives a video call from 'the CFO' asking for an urgent transfer. Both risks are measured below on digits, and section 5 sets them beside real, documented cases.

SYSTEM DIAGRAM



Three risks of generative models, each measured on digits in sections 2–4.

Today's part for the week's build

`day145_exercise.py` is today's parts (skewing a training set, training a detector, its probability that an image is generated, a 3 × 3 blur, and the share of flagged images that really are generated), checked by hand: Bayes' rule at 1% prevalence, the skew on twelve made-up ones, the blur of a single pixel, and a detector that learns to spot blurred digits.

ACTUAL OUTPUT

```
flagged precision at 1% prevalence: 0.1610 (by hand: 0.0095 / (0.0095 + 0.0495) = 0.1610)
skew on 12 made-up ones (3 leaning right): 4 ones kept, 3 leaning right (share 0.75); threes kept
    5 of 5
blur of one middle pixel: 9 pixels of 0.1111 (1/9 = 0.1111), total ink 1.0000; corner pixel: 4
    pixels, total 0.4444 (4/9 = 0.4444)
detector, real against blurred digits, 3 epochs: held-out real called generated 0.139, blurred
    called generated 0.806
```

Data: Day 126's 70,000 MNIST digits (training images 0–59,999). Day 143's measurements sort each digit into thirds by thickness and by lean, using cut points from all 60,000 training digits; Day 138's classifier names generated digits. The generative models are this week's, imported unchanged: Day 143's encoders, Day 144's text-conditioned diffusion model and guidance, Day 141's unconditioned model, Day 142's DDIM, with Days 129–130's VAE and Day 134's GAN as outside generators.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: what a detector's accuracy means when fakes are rare

DEFINITION — start here, no prior knowledge assumed

A detector has two error rates: the share of generated images it catches (true positive rate) and the share of real images it wrongly flags (false positive rate). What a user cares about is different: of the images it flags, how many really are generated? That depends on how common generated images are, the prevalence.

REAL-WORLD SCENARIO

A screening test for a rare disease: even an accurate test gives mostly false alarms when almost nobody has the disease, because the few true cases are outnumbered by the small share of mistakes among the many healthy people.

MATHEMATICS

$$\text{flagged that are generated} = TPR \cdot p \,/\, (TPR \cdot p + FPR \cdot (1 - p))$$

TPR = true positive rate, FPR = false positive rate, p = share of images that are generated (Bayes' rule)

IMPLEMENTATION

```
def flagged_precision(tpr, fpr, prevalence):
    return tpr * prevalence / (tpr * prevalence + fpr * (1 - prevalence))
```

ACTUAL OUTPUT

```
a detector that catches 95% of generated images (true positive rate) and wrongly flags 5% of real ones (false positive
    rate);
share of flagged images that really are generated = 0.95 p / (0.95 p + 0.05 (1 - p)), p = the share of images that are
    generated
       p  flagged that are generated  real images flagged per generated one caught
   0.500                      0.950                                          0.05
   0.100                      0.679                                          0.47
   0.010                      0.161                                          5.21
   0.001                      0.019                                         52.58
by hand for p = 0.01: 0.95 x 0.01 = 0.0095; 0.05 x 0.99 = 0.0495; 0.0095 / (0.0095 + 0.0495) = 0.161
Actual output from this lesson's run.
```

What it means

The same detector is excellent when half the images are generated (0.950 of flags correct) and poor when one in a hundred is (0.161): it then flags 5.21 real images for each generated one it catches, and at one in a thousand, 52.58. The hand calculation for p = 0.01 gives the same 0.161.

WHY CHOOSE THIS?

Two error rates and a prevalence are enough to say what a flag means; no experiment needed.

WHY NOT THIS? (TRADEOFFS)

Real prevalence is rarely known, and a detector's error rates measured on one kind of fake need not hold for another (section 3).

2. Bias: a skewed training set, and what the model makes of it

DEFINITION — start here, no prior knowledge assumed

The training ones are skewed on purpose: ones that do not lean right are removed at random until 75% of the remaining ones lean right (a third before). Day 143's encoders and Day 144's text-conditioned model are trained on this set, and the same training is repeated on the unskewed set as a control. Both models then draw ones and sevens at three guidance scales, and every drawing is measured.

REAL-WORLD SCENARIO

A photo library in which most pictures of nurses show women: a model trained on it may draw women for 'a nurse' even more often than the library shows them, and less common kinds of nurse may almost disappear from its output.

SPECIFICATION

Skew	ones not leaning right removed at random until 75% of the remaining ones lean right (a third before); every other digit kept
Models	Day 143's encoders and Day 144's text-conditioned model, trained exactly as on Day 144, on the skewed set; then again, same seeds, on all 60,000 digits as a control
Measured	400 samples per row from the same starting noise; only samples named as the asked-for digit count; 'leaning right', 'leaning left' and 'thick' use Day 143's cut points from the unskewed digits

IMPLEMENTATION

```
def skew_ones(y, lean, share=SKEW, seed=0):
    rng = np.random.RandomState(seed); idx = np.arange(len(y))
    one = y == 1; right = one & (lean == 1); other = one & (lean != 1)
    keep_other = int(right.sum() * (1 - share) / share)
    drop = rng.permutation(idx[other])[keep_other:]
    return np.setdiff1d(idx, drop)
```

ACTUAL OUTPUT

```
training set: the 60,000 training digits with 3,746 ones removed, all of them ones not leaning right;
ones leaning right: 0.333 of ones before, 0.750 after (2,996 ones left); every other digit unchanged
Day 143's encoders and Day 144's model (20,000 steps) trained on the skewed set: 714 s
generated: 400 samples per row from the same starting noise; 'leaning right' and 'thick' use Day 143's cut points
                prompt     w  named as asked  leaning right  leaning left   thick
                   one   1.0           0.955          0.414         0.024   0.408
                   one   2.0           0.995          0.638         0.003   0.784
                   one   4.0           1.000          0.618         0.000   0.995
                 seven   1.0           0.943          0.249         0.164   0.374
                 seven   2.0           0.998          0.228         0.211   0.749
                 seven   4.0           1.000          0.122         0.240   0.998
    a one leaning left   1.0           0.863          0.009         0.870   0.296
    a one leaning left   2.0           0.995          0.000         0.977   0.736
    a one leaning left   4.0           1.000          0.000         1.000   0.988
         training data                                 0.750         0.131   0.333   (ones for the lean columns, sevens for
             'thick')
control: the same encoders and model, same seeds, trained on all 60,000 training digits (no skew): 944 s
         one (no skew)   1.0           0.980          0.110         0.217   0.260
         one (no skew)   2.0           1.000          0.150         0.182   0.338
         one (no skew)   4.0           1.000          0.120         0.117   0.805
       seven (no skew)   1.0           0.917          0.229         0.199   0.346
       seven (no skew)   2.0           1.000          0.228         0.230   0.652
       seven (no skew)   4.0           0.995          0.334         0.186   0.947
         training data                                 0.333         0.333   0.333   (unskewed: ones for the lean columns,
             sevens for 'thick')
Actual output from this lesson's run.
```

What it means

The model does not copy its training shares even without a skew. The control, trained on all 60,000 digits, drew ones leaning right only 0.110 of the time and leaning left 0.217, against a third each in the data. So a skewed model has to be compared with the control, not with the data.

Against the control, the skew carried over and grew. Ones leaning right rose from 0.110 to 0.414 at w = 1 and from 0.150 to 0.638 at w = 2. The rarer kind shrank more than the data did: the skew left 0.131 of training ones leaning left (from a third, so about two fifths of the share remained), while the model's 0.217 fell to 0.024 at w = 1 (about a ninth) and to 0.003 at w = 2. Sevens, which were not skewed, came out about the same from both models (leaning right 0.249 and 0.229 at w = 1).

Guidance also exaggerated thickness, skew or not: sevens measured thick rose from 0.346 at w = 1 to 0.947 at w = 4 in the control, against a third in the data, as Day 144 found for ink. Asking explicitly overrode the skew: 'a one leaning left' drew ones leaning left 0.870 of the time at w = 1 and 0.977 at w = 2, from a kind that was only 0.131 of the training ones.

A first draft of this experiment had no control. Against the data alone the model seemed not even to copy the skew (0.414 leaning right against 0.750); the control showed that gap was the model's own drift towards upright ones, and that the skew had in fact been amplified.

DOCUMENTED CASE — Bloomberg on Stable Diffusion (2023)

Bloomberg (Nicoletti and Bass, 'Humans Are Biased. Generative AI Is Even Worse', June 2023) generated thousands of Stable Diffusion images for job titles and crime-related prompts and compared them with US job data. They reported almost three images of perceived men for each of perceived women, most occupations skewed male except low-paying ones such as housekeeper and cashier, and skin-tone disparities exaggerated beyond the data. Stability AI responded that all AI models have biases representative of the datasets they are trained on.

Bloomberg compared generated images with real job data. The digits add a second comparison: a model trained without the skew, which here drifted on its own. Against it, the skew in the data came out larger in the model, and larger still with guidance.

WHY CHOOSE THIS?

A control model trained without the skew separates what the skew did from what the model does anyway.

WHY NOT THIS? (TRADEOFFS)

It costs a second training run (944 s here), and for real data an unskewed version rarely exists; it has to be built or approximated.

3. Deepfake detection: a detector, other generators, and simple edits

DEFINITION — start here, no prior knowledge assumed

A detector is trained on real digits and on digits from section 2's model, then tested on held-out images of both kinds, on two generators it never saw (Day 129's VAE and Day 134's GAN), and on images changed by simple edits: a blur, added noise, fewer grey levels, and black and white.

REAL-WORLD SCENARIO

A shop assistant trained to spot one forger's banknotes: good at that forger's notes, but a different forger's, or a genuine note that has been through the wash, can fool them in either direction.

SPECIFICATION

Detector	Day 126's network [784, 128, 1], logistic loss, Adam lr 0.001, 10 epochs
Training	5,000 real training digits against 5,000 generated (prompts 'zero' ... 'nine', w = 2)
Tests	1,000 each: held-out generated, real test digits, VAE samples, GAN samples (10 epochs, as on Day 142), and the four edits applied to the held-out generated and real images; 'called generated' = probability above 0.5

IMPLEMENTATION

```
def train_detector(R, G, epochs=10, seed=0, batch=128, lr=1e-3):
    net = d126.init_net([784, 128, 1], seed); opt = d126.Adam(net, lr); rng =
        np.random.RandomState(seed)
    Xd = np.vstack([R, G]).astype(np.float32); yd = np.r_[np.zeros(len(R)), np.ones(len(G))]
    for _ in range(epochs):
        o = rng.permutation(len(Xd))
        for s in range(0, len(Xd) - batch + 1, batch):
            b = o[s:s + batch]; A = d126.forward(net, Xd[b], ["relu", "linear"])
            p = 1 / (1 + np.exp(-A[-1][:, 0].astype(np.float64)))
            grads, _ = d126.backward(net, A, ["relu", "linear"], ((p - yd[b]) / len(b))[:,
                None].astype(np.float32))
            opt.step(net, grads)
    return net
```

ACTUAL OUTPUT

```
detector: Day 126's network [784, 128, 1], logistic loss, 10 epochs on 5,000 real training digits and 5,000 digits from
section 2's model (prompts 'zero' ... 'nine', w = 2); 61 s including sampling
share called generated (probability above 0.5); 1,000 images each
                                images  called generated
          held-out generated (same model)             0.924
                       real test digits                0.048
                VAE samples (never seen)               0.041
                GAN samples (never seen)               0.061
                   generated, 3 x 3 blur               0.986
                        real, 3 x 3 blur               0.344
                generated, noise, sd 0.1               1.000
                     real, noise, sd 0.1               1.000
                generated, 4 grey levels               0.357
                     real, 4 grey levels               0.045
        generated, black and white (0.5)               0.219
             real, black and white (0.5)               0.041
if 1 image in 100 were generated: of the images this detector flags, 0.163 would be generated (true positive rate 0.924,
    false positive rate 0.048)
Actual output from this lesson's run.
```

What it means

On what it was trained for, the detector worked: 0.924 of held-out generated digits caught, 0.048 of real ones wrongly flagged. Even so, if 1 image in 100 were generated, only 0.163 of its flags would be right, as section 1 predicted.

Against generators it never saw it was useless: VAE samples were called generated 0.041 of the time and GAN samples 0.061, about as often as real digits. What it learned was specific to this one model, not 'generated' in general.

Simple edits broke it in both directions. Reducing generated digits to 4 grey levels or black and white lowered their detection from 0.924 to 0.357 and 0.219. Adding a little noise made it call every image generated, real ones included (1.000), and a blur made it flag 0.344 of real digits. A likely reason, not tested here: generated digits differ from real ones in fine pixel texture, which these edits overwrite.

DOCUMENTED CASE — A deepfake video call (Arup, Hong Kong, 2024)

In January 2024 an employee of the engineering firm Arup in Hong Kong joined a video call in which the other participants, including a senior executive, were deepfakes, and made 15 transfers totalling HK$200 million (about US$25.6 million) to five bank accounts. Arup confirmed that fake voices and images were used; Hong Kong police described it as the first case there with multiple deepfakes in one call. The section above suggests why spotting the fake is a weak defence on its own; confirming an unusual request through a second, independent channel does not depend on spotting it.

DOCUMENTED CASE — A cloned voice before an election (New Hampshire, 2024)

On 21 January 2024, the eve of the New Hampshire primary, robocalls with an AI-generated voice imitating President Biden told voters not to vote. The political consultant Steve Kramer admitted commissioning them; the US Federal Communications Commission fined him $6 million (proposed May 2024, finalised September 2024), and the carrier Lingo Telecom settled for $1 million. Kramer also faced criminal charges in New Hampshire.

WHY CHOOSE THIS?

A detector trained on a known generator is cheap and, on that generator, accurate.

WHY NOT THIS? (TRADEOFFS)

It does not transfer to other generators, simple edits defeat it, and when fakes are rare most of its flags are false alarms.

4. Memorisation: small training sets and copied digits

DEFINITION — start here, no prior knowledge assumed

Day 141's unconditioned model is trained on only 10, 100 and 1,000 digits, then draws 500 samples. For each sample, the pixel distance to its nearest training digit is compared with the same distance for 500 real test digits, which the model never saw: a sample much closer than any real new digit is a near copy.

REAL-WORLD SCENARIO

A student who has seen only ten example essays tends to hand in one of them with small changes; one who has read a thousand writes something of their own.

SPECIFICATION

Models	Day 141's: x_0 output, output layer started at zero; 8,000 steps each; first n of a fixed shuffle
Samples	500 each, Day 142's DDIM with 100 steps and no fresh noise
Near copy	closer than 2.0 to a training digit (pixel distance; Day 141's model on all 60,000 had its closest sample 1.37 away)

IMPLEMENTATION

```
for n in MEM_SIZES:
    t0 = time.perf_counter(); idx = order[:n]
    net, _ = d141.train_image_denoiser(X2[idx], MEM_STEPS)
    F = d141.to_pixels(d142.ddim_sample(d142.net_x0_fn(net), 500, np.random.RandomState(1), 100,
        0.0))
    dF, jF = d141.nearest_train(F, X[idx]); dT, _ = d141.nearest_train(Te, X[idx])
    out[n] = (F, X[idx][jF], dF)
    print(f"{n:6,d} {np.median(dF):16.2f} {np.median(dT):12.2f} {np.mean(dF < NEAR):12.3f}
        {np.mean(dT < NEAR):12.3f} "
          f"{len(np.unique(jF)):>11d} of {min(n, 500):<4d} {time.perf_counter() - t0:5.0f}s")
```

ACTUAL OUTPUT

```
Day 141's model, 8,000 steps, trained on the first n of a fixed shuffle of the training digits; 500 samples each (DDIM 100
    steps);
for each sample, the pixel distance to the nearest training digit; 500 real test digits give the distances an honest new
    digit
lands at; 'near copies': closer than 2.0. Day 141's model, trained on all 60,000, had its closest sample 1.37 away
     n  median: samples  test digits  near copies  test digits  different nearest   time
    10            0.45         8.13        0.806        0.000          10 of 10     323s
   100            4.45         6.93        0.196        0.000          70 of 100    300s
 1,000            4.35         5.72        0.028        0.022         209 of 500    288s
Actual output from this lesson's run.
```

What it means

With 10 training digits the model copied them: median distance 0.45 against 8.13 for real test digits, 0.806 of samples near copies, and all 10 of 10 training digits among the nearest. With 100, near copies fell to 0.196, spread over 70 of 100 training digits. With 1,000 they were 0.028, close to the 0.022 of real test digits that happen to lie that near a training digit.

Memorisation is not all-or-nothing: at 1,000 the samples were still closer to the training set (median 4.35) than real new digits were (5.72). Near copies are only the visible end of the effect.

DOCUMENTED CASE — Extracting training images from diffusion models (2023)

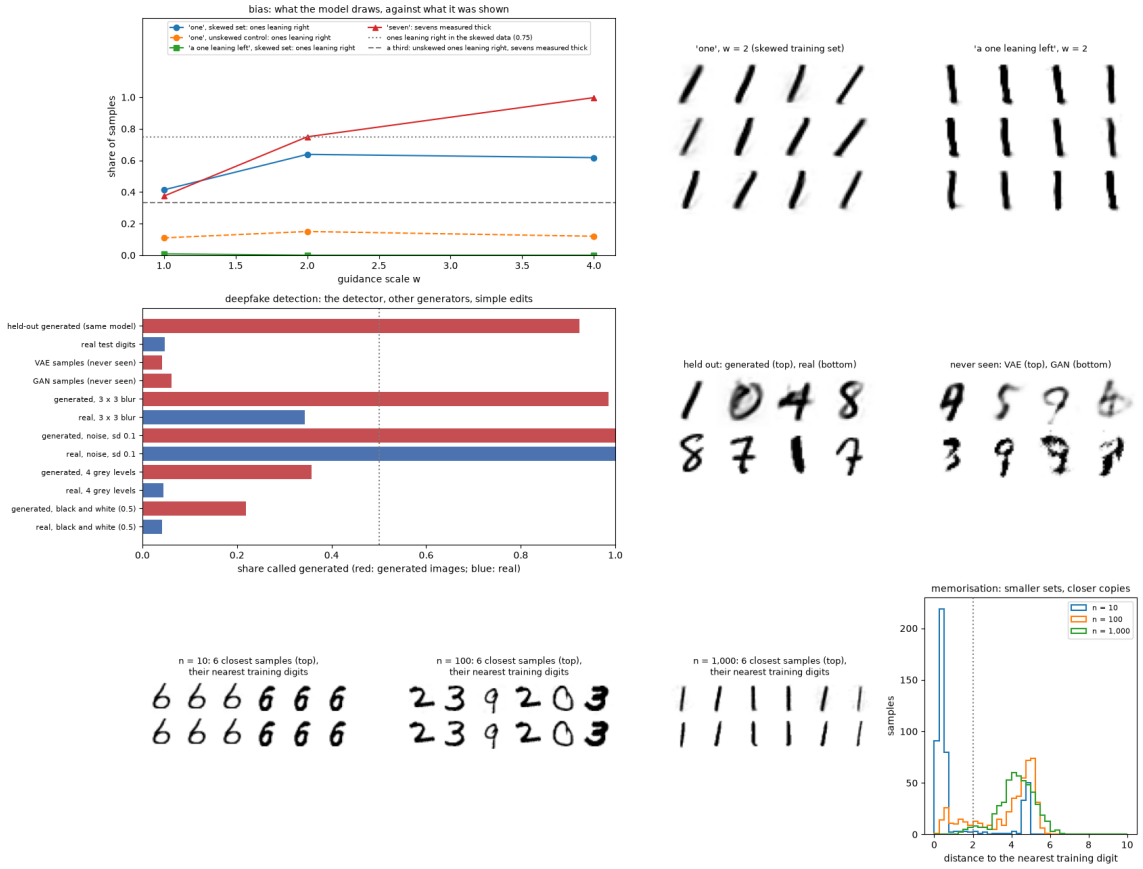
Carlini and colleagues ('Extracting Training Data from Diffusion Models', USENIX Security 2023) generated many images from Stable Diffusion and Imagen and filtered for near-identical repeats, extracting over a thousand training examples, from photographs of individual people to trademarked logos. The digits above show the same effect at a scale where it can be counted.

WHY CHOOSE THIS?

A nearest-neighbour check against the training set is simple and catches outright copies.

WHY NOT THIS? (TRADEOFFS)

It needs access to the training data, pixel distance misses copies that are shifted or recoloured, and a model can memorise a few images while generalising on most.



Top: ones leaning right against the guidance scale for the skewed model and the unskewed control, with sevens measured thick; 12 drawings each for 'one' and 'a one leaning left' at w = 2. Middle: the share called generated for each test set and edit; held-out and never-seen images. Bottom: for 10, 100 and 1,000 training digits, the 6 closest samples above their nearest training digits, and the distances of all 500 samples.

5. Four documented cases, and what the digits showed

Case	What happened	Measured today, on digits
Bloomberg, Stable Diffusion (2023)	Generated professionals skewed by gender and skin tone, beyond the job data	Ones leaning left: 0.131 of the skewed data; the model drew 0.024 (w = 1) against 0.217 for the unskewed control
Arup video call (2024)	Deepfaked colleagues on a call; HK$200 million transferred	A detector caught 0.924 of one model's fakes, 0.041 of another's
New Hampshire robocall (2024)	AI voice of a president told voters not to vote; $6 million fine	Edits defeated the detector: black and white lowered detection to 0.219
Carlini et al. (2023)	Over a thousand training images extracted from diffusion models	10 training digits: near copies 0.806

Sources: Bloomberg study as reported by Marketplace (marketplace.org, 14 June 2023) and the AI Incident Database (report 3180); Arup: CNN via ABC17 News (16 May 2024) and Construction Briefing; Carlini et al.: usenix.org and arXiv 2301.13188; New Hampshire: NPR (23 May 2024) and NHPR (26 September 2024).

Interview preparation — technical questions

Q: A detector has 95% true positive and 5% false positive rates. Is a flag reliable?

A: It depends on prevalence: at 1 generated image in 100, only 0.161 of flags are right (Bayes' rule).

Q: How would you test a deepfake detector honestly?

A: On generators and edits it never saw, not just held-out images from its training generator: here 0.924 held out, 0.041 on a VAE.

Q: How do you check a generative model for memorisation?

A: Nearest training image for each sample, against the same distance for real held-out images; here 0.806 near copies with 10 training digits, 0.028 with 1,000 (test digits 0.022).

Q: How would you measure bias in an image generator?

A: Many samples per prompt from fixed noise, an attribute measured on each, compared with the training data and with a control model trained without the skew.

Q: What practical defence works against a deepfake call?

A: Not tested here: confirming the request through a second, independent channel. Detectors alone were unreliable on new generators (0.041 on a VAE).

Interview preparation — theoretical questions

Q: Why can a model exaggerate a skew rather than copy it?

A: Measured here: the skew cut left-leaning ones to 0.131 of training ones, but the model's share fell from 0.217 (control) to 0.024, and to 0.003 with guidance. A likely reason, not tested here: the model favours the most typical drawing for a prompt, and guidance pushes further towards it, so rare variants shrink most.

Q: Why did the detector fail on the VAE and GAN?

A: Measured: it caught 0.924 of its own model's fakes but 0.041 of the VAE's. A likely reason, not tested here: each generator leaves different traces, and it learned only one model's.

Q: Why do small training sets memorise?

A: Measured: 10 digits gave 0.806 near copies, 1,000 gave 0.028. A likely reason, not tested here: with few examples, reproducing them is the easiest way to fit the training objective.

Q: Is a detector or a watermark the better defence?

A: Not tested here. A detector guesses after the fact; a watermark needs the generator's cooperation; neither helps against a model without one.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Each risk could be measured. A skew in the training data came out larger in the model and larger still with guidance; a control model, not the data alone, was needed to see it. A detector worked on the model it was trained against and failed on others and on simple edits, and at realistic prevalence most of its flags would be false. Small training sets were copied; larger ones much less. Measuring a risk is the first step to managing it; none of these numbers makes a model safe.

- Ones leaning right: 0.110 (control) to 0.414 (skewed, w = 1) and 0.638 (w = 2); leaning left 0.217 to 0.024 and 0.003.
- 'a one leaning left' still drew 0.870 leaning left at w = 1: asking explicitly overrode the skew.
- Detector: 0.924 held out, 0.041 VAE, 0.061 GAN; at 1% prevalence 0.163 of flags right.
- Edits: black and white 0.219; noise flagged every real image (1.000).
- Memorisation: near copies 0.806 (10 digits), 0.196 (100), 0.028 (1,000; test 0.022).

Suggested watch order, if starting from zero

- 1. Ethical Concerns with Generative AI (Part 1 of 2) (Douglas Schmidt) — the ethical concerns raised by generative AI, as a lecture.
- 2. How to spot deepfakes and AI-generated images (New Scientist) — how to spot deepfakes and AI-generated images, in practice.

Day 146 preview

- **Week 21 capstone**: this week's parts put together into one text-to-digit generator (encoders, captioned diffusion model, guidance), checked end to end against this week's numbers.